Advanced Computer Architecture CMP 511

Multiprocessor Operating System

Department of Computer Engineering

Pokhara University

Multiprocessor Operating System

- Multiprocessing operating system refers to the use of two or more processing units (i.e. CPU) with in a single computer system.
- We all know about dual core or quad core processor. All these processors share the same memory and the input output devices.
- In fact, all the processes are loaded into the main memory. Let us consider a two processor system. Now it is the operating system which assigns process to each CPU. Here each CPU works on the separate process which makes the processing very fast. After completing the execution of the processes, they are removed from the main memory.
- Again the OS assigns new process to the CPUs. Both the CPU now again executes the process which is assigned to them. This is done until all the processes are executed

BY DOING SO, THE SYSTEM WITH TWO CPUs WILL EXECUTE THE WORK TWO TIMES FASTER THAN THAT OF A SINGLE PROCESSOR SYSTEM

Classification of Multiprocessor OS

- Broadly, multiprocessor OS can be categorized as
  - Symmetric Multiprocessing
  - Asymmetric Multiprocessing
- Symmetric Multiprocessing
  - All the CPU has the equal right
  - There is no master slave configuration
  - Each CPU has the right to execute any process
  - Each CPU can access the memory, input-output devices or any resources
  - In short, one operating system controls all CPUs and all the CPUs have equal rights

Classification of Multiprocessor OS

- Asymmetric Multiprocessing
  - There is a master slave architecture present instead of giving all rights to all processor
  - Instead of giving all rights to all processor, we can make one as a mater and other as a slave
  - Master CPU decides which CPU will perform which task, it means master executes system programs and slaves are used for application programs
- Many a times, if all processors access input output devices or memory or any other hardware, the designing of such connections get(s) difficult and the production cost for circuit design also gets high.
- So to prevent this, one processor is assigned for all interaction with input-output devices or memory or any other hardware devices.
- And all other processors are used to improve the processing speed; so in any architecture, where all CPU don have similar rights, it is called **asymmetric multiprocessing.**

Advantages of Multiprocessor OS

- **Increased Throughput**
  - It is the amount of work completed in a unit time
  - So when we have multiple processors available, then we will complete more work in less time.
- **Increased Stability**
  - If some processor gets crashed, then the processing will not be stopped; since other processors are still running; reliability is said to be increased in multiprocessing
- **Increased Saving**
  - If we are using a quad core processor, then we are doing the same amount of work, that we do with four single computers.
- **Parallel Processing**
  - In Multi-Programming and Multi-Tasking, we saw that processor works so fast that we feel like all processes are running parallel, but it works like one process at a time.
  - Since multiple processors work on multiple processes at the same time, the multiprocessing OS gives parallel processing

Classification of Multiprocessor OS

- Advanced architectures require specialized OS designs for efficiency and scalability.
- These systems optimize resource management, reliability, and parallel execution.
- Basic Classification includes:
  - Master-Slave OS
  - Separate Supervisor System
  - Floating Supervisor System
  - Distributed OS
  - Parallel OS
  - Network of Workstations (NOW)

Master-Slave OS

- A single processor (master) controls all other processors (slaves).
- The master assigns tasks, and slaves execute them without decision-making.
- Example: Early IBM mainframe systems.
- Advantages:
  - Simpler task management
  - Suitable for specialized systems
- Disadvantages:
  - Single point of failure
  - Limited scalability

Separate Supervisor System

- A dedicated processor exclusively handles system supervision tasks.
- Other processors handle user applications.
- Example: Used in certain mainframes and real-time systems.
- Advantages:
  - Improved system security and stability
- Disadvantages:
  - Adds overhead for inter-processor communication

Floating Supervisor System

- No fixed processor acts as a supervisor.
- Any processor can take control based on system needs.
- Example: Used in fault-tolerant and adaptive computing systems.
- Advantages:
  - Higher fault tolerance
  - Load balancing capabilities
- Disadvantages:
  - Complexity in managing supervisor transitions

Distributed Operating Systems

- Manages multiple processors across different machines as a single system.
- Enables seamless resource sharing and process synchronization.
- Example: Google Cloud, Hadoop
- Advantages:
  - High scalability
  - Efficient resource utilization
- Disadvantages:
  - Complex system design
  - Requires robust network infrastructure

Parallel Operating Systems

- Designed for tightly coupled multiprocessor systems.
- Provides parallel execution of processes across multiple processors.
- Example: Parallel versions of Linux, Windows HPC
- Advantages:
  - High-speed processing
  - Ideal for scientific and engineering computations
- Disadvantages:
  - Requires specialized hardware support
  - Complexity in process synchronization

Network of Workstations (NOW)

- A cluster of independent computers connected via a network.
- Uses distributed computing techniques to share tasks.
- Example: Beowulf clusters, Grid Computing
- Advantages:
  - Cost-effective alternative to supercomputers
  - High availability and fault tolerance
- Disadvantages:
  - Dependent on network performance
  - Requires distributed resource management software

Comparison Table

- OS for advanced architectures cater to specific system needs.
- Selection depends on performance, fault tolerance, and scalability requirements.
- Future trends focus on hybrid models integrating distributed and parallel processing.

Feature	Master-Slave OS	Separate Supervisor	Floating Supervisor	Distributed OS	Parallel OS	NOW
Central Control	Yes	Yes	No	No	No	No
Scalability	Low	Moderate	High	High	High	High
Fault Tolerance	Low	High	High	High	Moderate	High
Examples	IBM Mainframes	Real-time Systems	Fault-tolerant OS	Google Cloud	HPC Systems	Grid Computing

Software requirements of Multiprocessor OS

- Hypervisor is a piece of software or firmware that creates and run **virtual machine**.
- Virtualization is the technology of splitting a physical resource into as many logical resource as we want.
- Virtualization is a technology that transforms hardware/physical into software/software
- Virtualization is not only about splitting of resources, it also means of adding resources. So created multiple system is then call logical system or virtual machine
- A hypervisor is also sometimes called a Virtual Machine Manager (VMM)

Software requirements of Multiprocessor OS

- **Case Study** –Why IT companies require Virtual Machine?
- **Data Center Setup requirement** – US based clothing company
  - 4000 square feet data center
  - 200 physical server
  - Cooling system for 200 severs
  - Physical disk for server
  - Cabling, internet connectivity, power whips
  - Staffs
  - Electricity and power back up
- **Solution:**
  - In order to have increased revenue, either one have to adopt cloud computing, or adopt pay per use model, adopt virtualization or virtual machine.

Software requirements of Multiprocessor OS

- **Type 1 – Hypervisor**
  - Also called bare metal hypervisor
  - Above the physical server is the virtualization serve named hypervisor.
  - Type 1 hypervisor runs directly on the system hardware
  - Type 1 hypervisor is called firmware
    - Hardware update ➔ firmware update
    - Software update ➔ android update
  - A guest OS runs on another level above the hypervisor
  - VMware ESXi is a type 1 hypervisor that runs on the host server hardware without an underlying OS
  - Type 1 hypervisor act as their own operating system
  - ESXi provides a virtualization layer that abstracts the CPU, storage, memory and networking resources of the physical host into multiple virtual machine
  - VMware vSphere is a virtualization platform that's part of a suite of VMware products. The suite includes tools for managing and deploying virtual environments, including virtual machines (VMs) and virtual appliances.

Software requirements of Multiprocessor OS

- **Type 2 – Hypervisor**
  - Hypervisor that runs within a conventional OS environment and the host OS provider
  - It is used for testing purpose, learning purpose
  - Example of Type 2 hypervisor are VMware workstation, Microsoft VirtualPC, Oracle VirtualBox
  - It does not have direct access to the host hardware and resources
  - Type 2 has dependency on host OS

Software requirements of Multiprocessor OS

Criteria	Type 1 Hypervisor	Type 2 Hypervisor
Known as	Bare metal and native	Hosted
Virtualization	Hardware level virtualization	OS level virtualization
Operation	Guest OS and application run on the hypervisor	Run as an application on the host OS
Scalability	Better Scalability	Not so much because of its resilience on the underlying OS
System Interdependence	Host direct access to the hardware along with VM it host	Are not allowed to directly access the host hardware ad its resources
Performance	High performance as there is no middle layer	Comparatively has reduced performance rate as it runs with extra overhead
Security	More secure	Less secure, as any problem on the base OS affects the entire system
e.g.	VMware ESXi, Microsoft Hyper-V, and Citrix XenServer are all popular hypervisors that manage virtual machines (VMs).	Vmware worksation Microsoft VirtualPC Oracle VirtualBox

Software requirements of Multiprocessor OS

- Multiprocessor OS manages multiple CPUs in a system.
- Software requirements ensure efficient task scheduling, memory management, and process synchronization.
- **Some more key requirements include:**
  - Process Management
  - Memory Management
  - Synchronization & Communication
  - Fault Tolerance & Security
  - Implicit and Explicit Parallelism

Software requirements of Multiprocessor OS

- **Process Management**
  - Efficient CPU scheduling for multiple processors.
  - Load balancing and task allocation strategies.
  - Support for parallel execution.
- **Memory Management**
  - Shared memory access control.
  - Dynamic allocation and deallocation.

Software requirements of Multiprocessor OS

- **Synchronization & Communication**
  - Mechanisms like semaphores, mutexes, and message passing.
  - Prevention of deadlocks and race conditions.
- **Fault Tolerance & Security**
  - Redundancy and error handling for fault tolerance.
  - Protection against unauthorized access and security threats.

Software requirements of Multiprocessor OS

- **Parallelism in Multiprocessor**
  - **Implicit Parallelism:**
    - OS automatically manages parallel execution.
    - Example: Multi-threaded applications using parallel compilers.
  - **Explicit Parallelism:**
    - Programmers manually define parallel execution.
    - Example: OpenMP, MPI-based distributed processing.

Requirement of operating system implementation

- The Operating System (OS) is a critical component that manages hardware and software resources.
- Essential requirements of an OS include:
  - Role of OS
  - Modular Design
  - Scalability
  - Portability
  - Kernel Space and User Space

Role of the Operating System

- Acts as an interface between users and hardware.
- Manages system resources efficiently.
- Key functions:
  - Process and memory management
  - File system management
  - Device management
  - Security and user authentication

Modular Design

- OS components are structured as independent modules.
- Benefits:
  - Easier maintenance and updates
  - Enhanced flexibility and adaptability
- Example: Microkernel architecture separates core functions into smaller, independent modules.

Scalability

- The OS should efficiently handle increasing workloads.
- Required for:
  - Multiprocessor and distributed systems
  - Cloud computing and high-performance computing
- Examples:
  - Linux and Windows Server support scalable architectures.

Portability

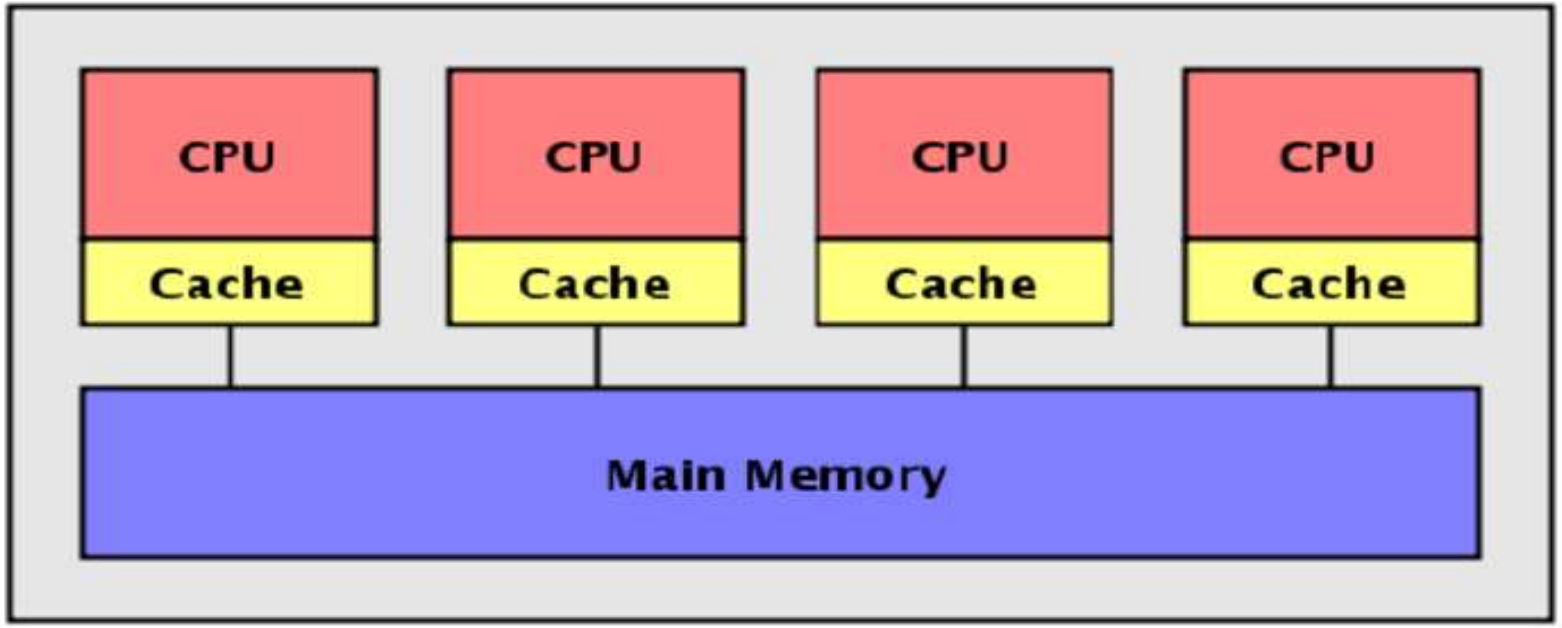
- Ability to run on different hardware architectures with minimal changes.
- Ensures OS can be adapted across various devices (desktops, servers, mobile, embedded systems).
- Example: Unix-based systems like Linux and macOS are highly portable.

Kernel Space vs. User Space

- **Kernel Space:**
  - Core part of the OS with full access to hardware.
  - Handles process scheduling, memory management, and device drivers.
- **User Space:**
  - Runs applications and interacts with the OS via system calls.
  - Provides isolation for user programs, improving security and stability.
- **In conclusion:**
  - An OS must be modular, scalable, portable, and structured efficiently.
  - Proper design ensures optimal performance and adaptability.
  - Future OS advancements focus on AI-driven automation, security enhancements, and cloud-native designs.

PThread in shared memory systems – Case Study

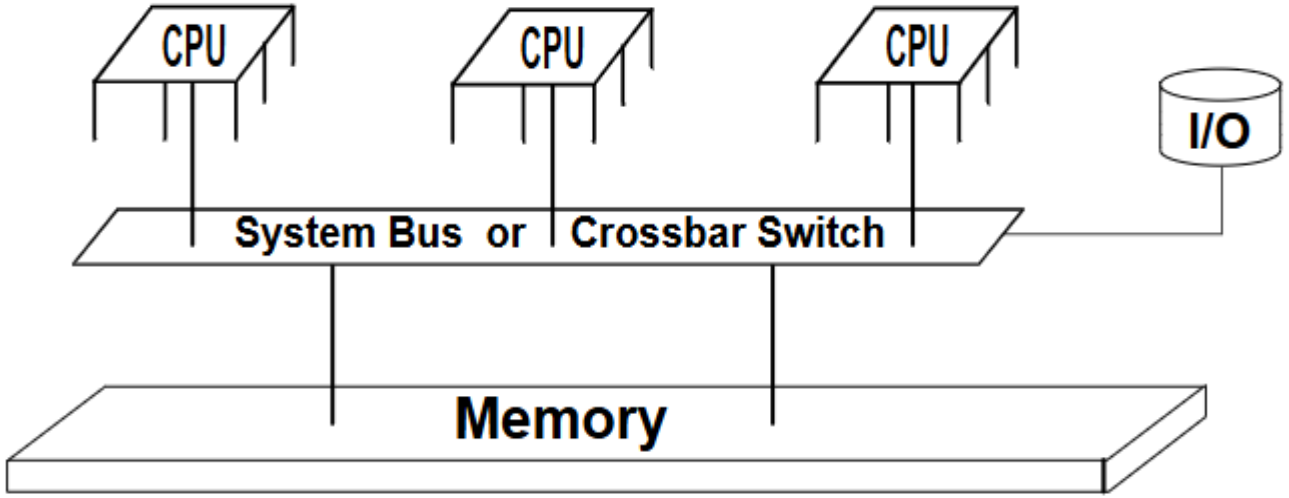
- **Shared memory** is a concept where two or more process can access the common memory
- Communication is done via this shared memory where changes made by on process can be viewed by another process
- Devices exchange information by writing to and reading from a pool of shared memory



Shared memory architecture

Shared Memory Programming

- In computer science, shared memory is a memory that may be simultaneously accessed by multiple programs with the intent to provide communication among them or avoid redundant copies.
- Using memory for communication inside a single program, e.g. among its multiple threads, is also referred to as shared memory.
- Shared memory is an efficient means of passing data between programs. Depending on context, programs may run on a single processor or multiple separate processors.



Shared Memory Model

- Program consists of threads of control with
  - Shared variables
    - Accessible by everyone in the memory
  - Private variable
    - Can only be accessed by the person who owns them
- Threads communicate implicitly by writing and reading shared variables
- Threads coordinate by synchronizing on shared variables
- Threads can be dynamically created and destroyed
- Other programming models includes: distributed memory, hybrid, data parallel programming model (single thread of control), shared address space etc

Threads

- Threads of the same process share memory space; i.e. they access the same chunk of memory with the same address – Threading represents the OS support for shared memory programming.
  - All the threads share the same memory context (code and data). There is no memory protection between threads in the same process
  - Any thread may alter any data item. Each thread has its own register context (including PC and SP), and its own stack.
- Advantages of Threads:
  - Shared memory space
    - Unlike separate processes, threads within a process share the same address space, allowing them to read and write to the same memory locations directly.
  - No IPC required
    - Since threads already have access to the shared data, they don't need to use complex inter-process communication methods like message passing to exchange information
  - Threads can take advantage of multiprocessors
    - When a program uses multiple threads, each thread can be assigned to a different CPU core on a multiprocessor system, allowing them to run simultaneously
  - There is a higher throughput when multiple threads cooperate in a single job, the whole process does not stop if one thread must wait
  - Threads are cheap to create as they only need a stack and storage for registers
    - Threads use very little resources of an operating system in which they are working. That is, threads do not need new address space, global data, program code or operating system resources.
  - Context switching becomes fast when working with threads
    - As the threads share the same memory space, the switching of the threads is faster as less information needs to be saved and restored. There is no need to change the memory maps during the switching, as the threads share data and resources like memory. Switching between threads of a single process can be faster than between two separate processes because threads share the same virtual memory maps, so a TLB flush is not necessary. The time to switch between two separate processes is called the process switching latency.

Threads can be implemented in two different ways:

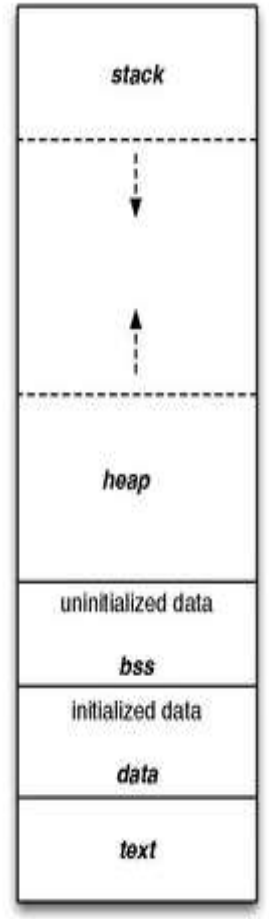
1. By the use of library routines (e.g. Pthreads), independent of the operating system(i.e. the operating system manages a single process; the process manages its own threads)
    - To achieve thread management within a single process using library routines like Pthreads, independent of the operating system's thread management, you would implement a user-level threading library where the application itself is responsible for scheduling and managing thread context switches, essentially creating an abstraction layer on top of the OS process that simulates multithreading behavior within the process's address space.
2. By adding threads facility to the operating system. System services are required for creating a new process (initially consisting of a single thread). Creating a new thread within an existing process.
3. The first approach is simpler, and minimizes the overhead involved in a context switch between threads (since no system call is involved).
4. The second approach, however allows one thread in a process to block itself on an IO operation while allowing the other threads to continue executing. For this reason, some system make both approaches available

Multi-Threads

1. In a multithreaded process, all threads within the same process share the same memory space and open files, but each thread has its own unique stack, instruction pointer, and set of registers within that shared memory area, allowing them to execute independently while accessing common data.
2. In a multithread-aware operating system, the data structure used to store information about each individual thread is called a Thread Control Block (TCB), which functions similarly to a Process Control Block (PCB) but specifically manages thread-related details like its unique thread ID, register values, and stack pointer, allowing the OS to effectively manage multiple threads within a single process.
3. An operating system had to keep track of processes, and store its per process information in a data structure called a process control block. A multithread-aware system also needs to keep track of threads.

Multi-Threads

- The items that the operating system must store inside PCB that are unique to each other thread are:
    a. Thread ID
    b. Saved registers, stack pointer, instruction pointer
    c. Stack (local variables, temporary variables, return addresses)
    d. Signal mask
    e. Priority (Scheduling Information)
- The items that are shared among threads within a process are:
    a. Text segments (instructions)
    b. Data segment (static and global data)
    c. BSS segment (uninitialized data)
    d. Open file descriptors
    e. Signals
    f. Current working directory
    g. User and group IDs



This shows the typical layout of a simple computer's program memory with the text, various data, and stack and heap sections.

- **Thread ID:**
  - A unique identifier for each thread, allowing the operating system to distinguish between different threads within a process.
- **Saved registers:**
  - The values of the CPU registers that were active when a thread was interrupted, which need to be restored when the thread resumes execution.
- **Stack pointer:**
  - A memory address that points to the top of the thread's stack, where local variables and temporary data are stored.
- **Instruction pointer:**
  - The address of the next instruction the thread needs to execute.
- **Stack (local variables, temporary variables, return addresses):**
  - The memory area allocated to each thread where its local variables and temporary data are stored, including return addresses from function calls.
- **Signal mask:**
  - A set of signals that are currently blocked for the thread, determining which interrupts can be received.
- In Conclusion, while multiple threads within the same process share the same memory space and code, each thread has its own unique stack and set of register values, which is why these elements are considered unique to each thread.

Level of Threads:

- Threads are implemented in following two ways-
  a. User Level Threads
  b. Kernel level Threads

a. User Level Threads

- Many-to-one threads mapping
- Implemented by user level run-time libraries
  - Create, schedule, synchronize threads at user level.
- OS is not aware of user-level threads
  - OS think reach process contains only a single thread of control
- Advantages
  - Don’t require OS support; Portable/**secured**
  - Can tune scheduling policy to meet application demand
  - Lower overhead thread operations since no system calls
- Disadvantages
  - Cannot leverage multiprocessors
  - Entire process blocks when one thread block

Level of Threads

b. Kernel Level Threads
   - One-to-one threads mapping
   - OS provide each user-level thread with a kernel thread
   - Each kernel level thread scheduling independently
   - Thread operations (creation, scheduling, synchronization) performed by OS
   - Advantages
     - Each kernel level thread can run in parallel on a multiprocessor
     - When one thread blocks, other threads from process can be scheduled
   - Disadvantages
     - Higher overhead for thread operations
     - OS must scale well with increasing number of threads

Pthreads in Shared Memory Programming

- POSIX Threads, usually referred to as pthreads
- Widely used UNIX specification
- A standard for Unix-like operating systems
- A library that can be linked with C programs
- Specifies an application programming interface (API) for multi-threaded programming
- POSIX is the Portable Operating System Interface, the open operating interface standard accepted world-wide. It is produced by IEEE and recognized by ISO and ANSI
- The Pthreads API is only available on POSIX complaint systems like Linux, MacOS X, Solaris, HPUX etc.

Pthreads in Shared Memory Programming

- **Meaning of "POSIX":**
  - "POSIX" stands for "Portable Operating System Interface," which is a standard set by the IEEE (Institute of Electrical and Electronics Engineers) that aims to provide a consistent programming interface across different Unix-like operating systems.
- **Functionality:**
  - Pthreads provides a set of functions for creating, managing, and synchronizing threads within a program, enabling parallel execution of tasks.
- **Benefits:**
  - By using pthreads, developers can write multi-threaded applications that can run on various Unix-like operating systems without significant code changes, promoting portability.
- **Implementation:**
  - Most Unix-like operating systems provide a "pthread" library that can be linked with C programs to access the POSIX Threads functionality

Pthreads APIs

- Pthreads are defined as a set of C language programming types and procedure calls. Vendors usually provide a Pthreads implementation in the form of a header/include file and a library that you link with your program. The primary motivation for using Pthreads is to realize potential program performance gains
- Thread execution management APIs
  - Function start with “pthread”
- Thread property (attributes) management APIs
  - Function start with “pthread_attr”
- Synchronization APIs
  - Mutex: “pthread_mutex_” and “pthread_mutexattr_”
  - Conditional varibales: “pthread_cond_” and “pthread_condattr_”
  - Read/write locks: “pthread_rwlock_”
  - Barriers: “pthread_barrier_”
  - Spin Locks: “pthread_spin_”
  - Thread specific data
- Thread specific data
  - Functions start with “pthread_key_”

Why Pthreads?

- Lightweight
  - Typically, the cost of creating and managing a process is higher than that of a thread, and therefore, threads can be created with less OS overhead. Also, threads require fewer system resources than processes
    - Lightweight threads aren't tied to a particular platform thread and don't come with the huge amount memory per-allocated for them. They're scheduled and managed by the runtime, not the underlying OS. That's why a significantly larger number of lightweight threads can be created
- Efficient Communication/Data Exchange:
  - When compared to using MPI (message passing interface) libraries for on-node communication, using Pthreads may become a huge benefit over achieving a better performance.
  - Both Pthreads and MPI are used for parallel programming, the key difference is that Pthreads manages threads within a single process on a single machine, whereas MPI manages processes across multiple machines in a distributed memory environment, making Pthreads ideal for fine-grained parallelism on a single node and MPI for large-scale distributed computing across clusters.
    - In parallel computing, "fine-grained parallelism" refers to breaking down a program into many small, independent tasks, while "coarse-grained parallelism" involves dividing the program into a smaller number of larger tasks, essentially meaning that fine-grained parallelism has smaller units of work with more frequent communication between processors, whereas coarse-grained parallelism has larger units of work with less frequent communication.

Lock

"Mutual exclusion" refers to a mechanism in computer programming that ensures only one thread can access a shared resource at a time, effectively preventing multiple threads from modifying the data simultaneously; essentially, it "locks" the resource, making other threads wait until the current thread releases the lock and the resource becomes available again, thus allowing access by "at least one thread at a time.".

Key points about mutual exclusion:

- **Purpose:**
  - **To avoid race conditions where multiple threads try to access and modify shared data at the same time, potentially leading to inconsistent results.**
- **Implementation:**
  - Typically achieved using a "mutex" (mutual exclusion lock), which is a synchronization primitive that a thread acquires before accessing the shared resource and releases when finished.
  - Allow access by at least one thread at a time
- **Critical section:**
  - The part of the code where a thread accesses the shared resource is called the "critical section".
  - Defined by the programmer (compiler can’t figure this out)
- **Blocking behavior:**
  - When a thread attempts to access a resource already locked by another thread, it is "blocked" (put into a waiting state) until the lock is released.
  - “Mutual Exclusion”
  - Make others block (wait) until the resource is free
  - Called a **mutual-exclusion lock** or just **lock**, for short

Lock ADT

- Lock as an ADT with operation
  - new: make a new lock, initially “not held”
  - acquire: blocks if this lock is ready currently “held”
    - acquire allows a thread to take ownership of a lock. If a thread tries to acquire a lock currently owned by another thread, it blocks until the other thread releases the lock. At that point, it will contend with any other threads that are trying to acquire the lock.
  - release: makes this lock “”not held”
    - If>=1 threads are blocked on it, exactly one will acquire it
      - Allow access to at most one thread at a time
      - Only one thread can hold a lock at a time. If a thread tries to take a lock that is already held by another thread, then it must wait until the lock is released. When this happens, there is so called “contention” for the lock.

```
class BankAccount {
   private int balance = 0;
   private Lock lk = new Lock();
   …
   void withdraw(int amount) {
      lk.acquire(); // may block
      int b = getBalance();
      if(amount > b)
         throw new WithdrawTooLargeException();
      setBalance(b – amount);
      lk.release();
   }
   // deposit would also acquire/release lk
}
```

Semaphores

- A semaphore is a signaling mechanism, and a thread that is waiting on a semaphore can be signaled by another thread
- Semaphores are primarily used to coordinate the execution of multiple threads or processes accessing shared resources, preventing race conditions and ensuring only one thread can access a critical section at a time.
- It uses two operations
  - **Wait**
    - When a thread wants to access a resource, it performs a "Wait" operation on the semaphore. If the semaphore value is positive, the thread is allowed access and the value is decremented; if the value is zero, the thread is blocked until another thread performs a "Signal" operation.
  - **Signal for the process synchronization**
    - When a thread finishes using a resource, it performs a "Signal" operation, which increments the semaphore value, potentially allowing a waiting thread to proceed.
- A semaphore either allows or disallows access to the resource, which depends on how it is set up
- **Characteristics of Semaphores**
  - It is a mechanism that can be used to provide synchronization of tasks
  - It is a low-level synchronization mechanism
  - Semaphore will always hold a non-negative integer value
  - Semaphore can be implemented using test operations and interrupts, which should be executed using file descriptors

Types of Semaphores

- The two types of semaphores are
  a. **Counting semaphores**
     - Can control access to a pool of resources where multiple threads can access the resource simultaneously up to a certain limit (value can be any non-negative integer).
  b. **Binary Semaphore:**
     - Only allows one thread access to a resource at a time, acting like a simple lock (value can only be 0 or 1).

Example of Semaphore

- The below given program is a step by step implementation, which involves usage and declaration of semaphore

```
Shared var mutex: semaphore = 1;
Process i
    begin
P(mutex);
execute CS;
V(mutex);
.
.
End;
```

Task6

1. Compare and contrast Symmetric Multiprocessing (SMP) and Asymmetric Multiprocessing (AMP) in terms of process scheduling, resource sharing, and fault tolerance. Provide examples of real-world implementations.
2. Explain the role of POSIX threads (PThreads) in shared memory systems. How does PThread synchronization (mutexes, condition variables) ensure safe and efficient parallel execution in a multiprocessor environment?
3. <u>Recommended Reading</u>
   - Distributed Scheduler – PVM (A Case Study) p. 425)
     - *Describe the working of PVM (Parallel Virtual Machine) as a distributed scheduler. How does PVM manage task allocation and communication across heterogeneous computing nodes? Illustrate with a practical scenario.*
   - Pthreads – (A Case Study)
     - *Shared memory, threads – implementations, user level, kernel level, multithreads, PCB, TCB, Pthreads in shared memory programming, Pthreads APIs, Why Pthreads, Semaphores - types*